INHA UNIVERSITY TASHKENT
FALL SEMESTER 2024

SOC 3010
OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010
OPERATING SYSTEM

WEEK 4 LECTURE 2

MEMORY ADDRESSING

PAGING UNIT

Paging in Hardware

Paging in Hardware

- The paging unit translates linear addresses into physical ones.
- It checks the requested access type against the access rights of the linear address.
- If the memory access is not valid, it generates a page fault exception.
- For the sake of efficiency, linear addresses are grouped in fixed-length intervals called pages; contiguous linear addresses within a page are mapped into contiguous physical addresses.
- In this way, the kernel can specify the physical address and the access rights of a page instead of those of all the linear addresses included in it.

Paging in Hardware

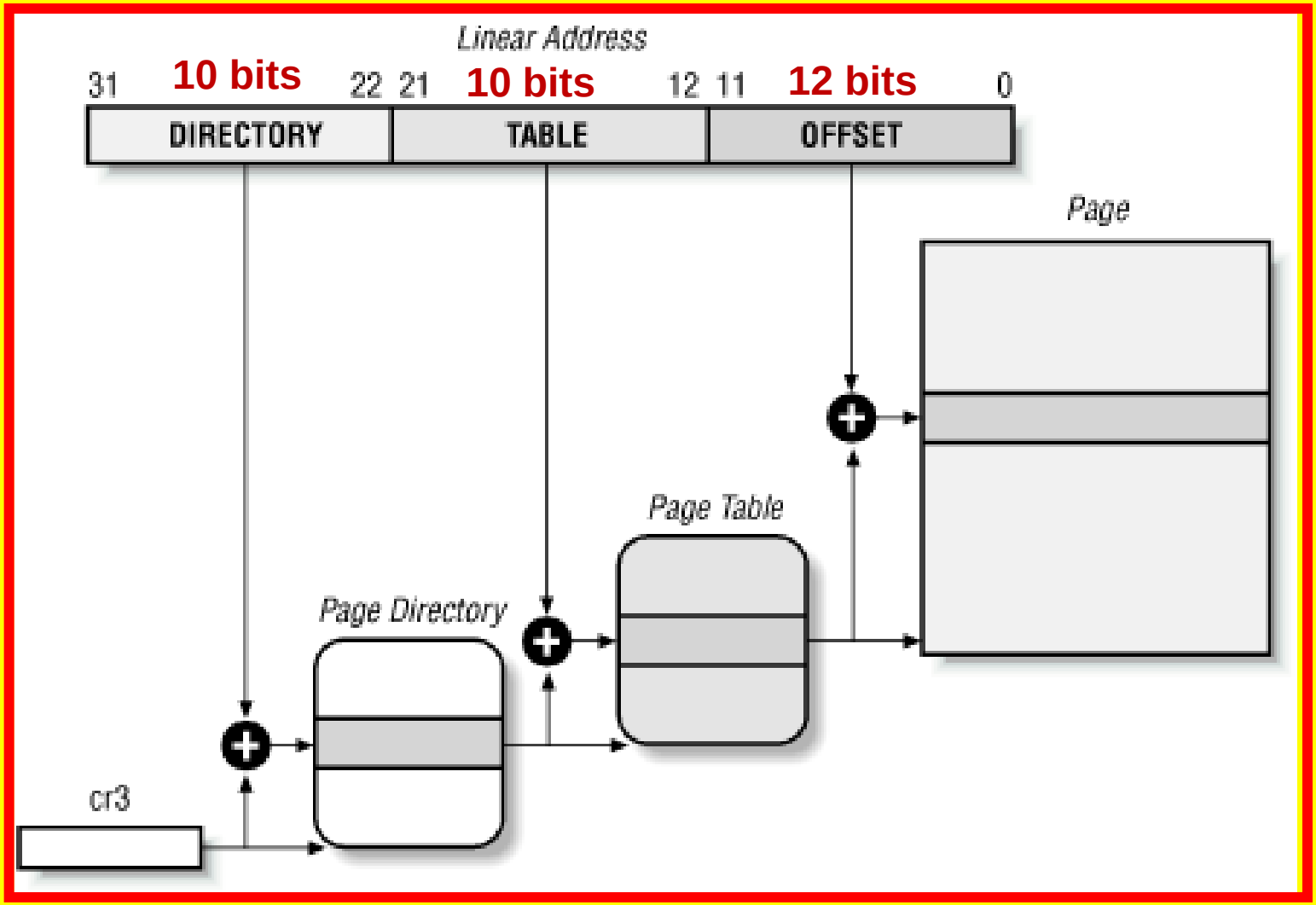
- Following the usual convention, we shall use the term **"page" to refer both to a set of linear addresses and to the data contained in this group of addresses.**
- **The paging unit thinks of all RAM as partitioned into fixed-length page frames (they are sometimes referred to as physical pages).**
- **Each page frame contains a page, that is, the length of a page frame coincides with that of a page.**
- **A page frame is a constituent of main memory, and hence it is a storage area.**
- **It is important to distinguish a page from a page frame: the former is just a block of data, which may be stored in any page frame or on disk.**
- **The data structures that map linear to physical addresses are called page tables; they are stored in main memory and must be properly initialized by the kernel before enabling the paging unit.**
- **In Intel processors, paging is enabled by setting the PG flag of the cr0 register.**
- **When PG = 0, linear addresses are interpreted as physical addresses.**

Paging in Hardware

- **Regular Paging**
- Starting with the i80386, the paging unit of Intel processors handles 4 KB pages.
- The 32 bits of a linear address are divided into three fields:
  - Directory
    - The most significant 10 bits
  - Table
    - The intermediate 10 bits
  - Offset
    - The least significant 12 bits
- The translation of linear addresses is accomplished in two steps, each based on a type of translation table.
- The first translation table is called Page Directory and the second is called Page Table.
- The physical address of the Page Directory in use is stored in the cr3 processor register.
- The Directory field within the linear address determines the entry in the Page Directory that points to the proper Page Table.
- The address's Table field, in turn, determines the entry in the Page Table that contains the physical address of the page frame containing the page.
- The Offset field determines the relative position within the page frame. Since it is 12 bits long, each page consists of 4096 bytes of data.

Paging in Hardware

- **Paging by Intel 80x86 processors – TWO LEVEL PAGING**



Paging in Hardware

- **TWO-LEVEL PAGING UNIT**

Consider two-level Paging unit using a 32-bit linear address divided into three fields - Page directory field containing most significant 10 bits, Page Table field containing intermediate 10 bits and page offset field containing least significant 12 bits.

(I) Determine the following :

Total number of entries in Page directory Table

Total number of entries in each Page Table :

Total number of Page tables that can be stored :

Size of each page :

Total number of pages that can be stored :

Total amount of main memory required to store these pages:

(ii) Determine the maximum amount of memory required to store all pages, Page directory & Page tables assuming that each entry in the page directory and page tables take 32-bits (each entry size =32-bits).

Memory required to store 1 Page Directory Table =

Memory required to store 1024 Page Tables =

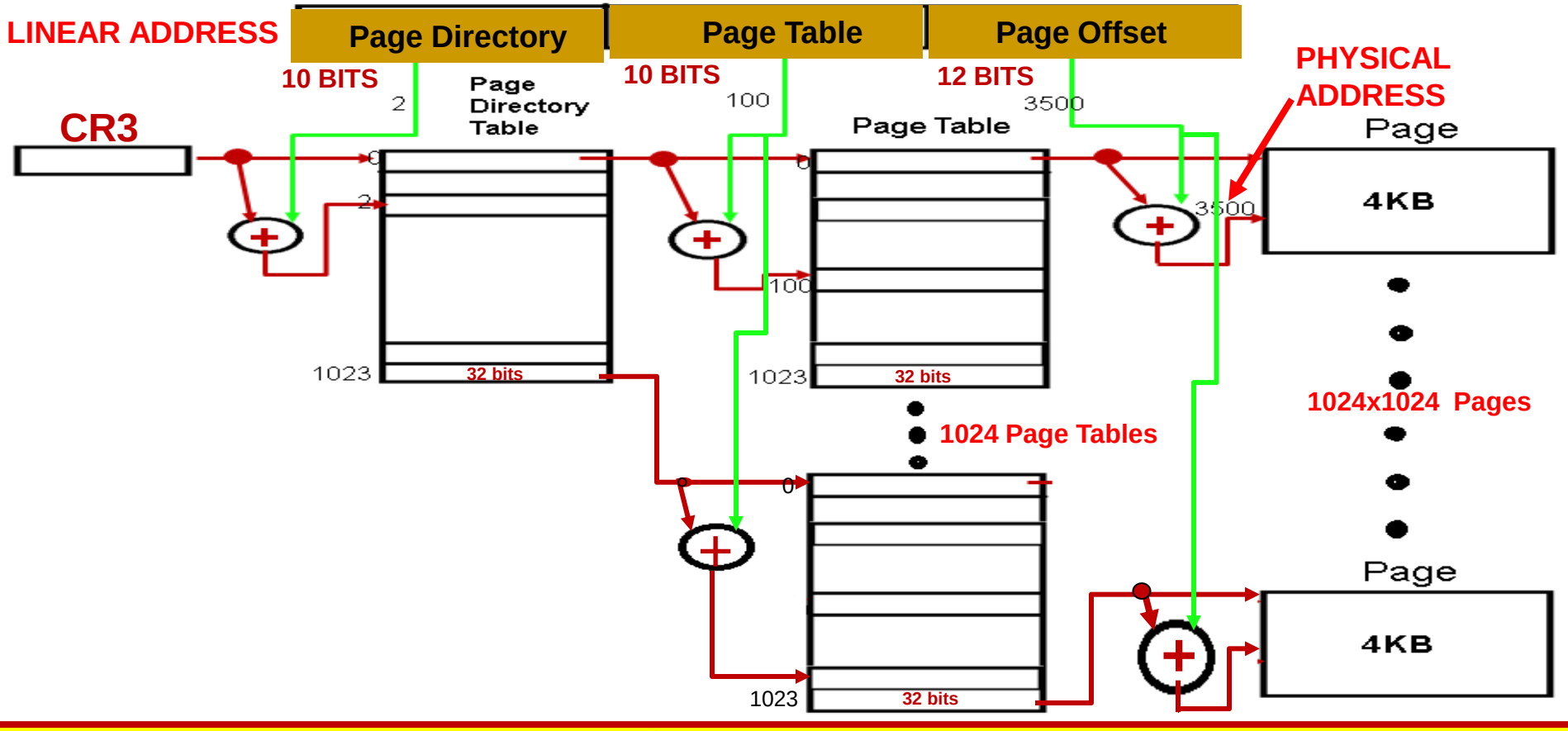
Memory required to store all Pages =

Maximum amount of memory required =

Paging in Hardware

TWO-LEVEL PAGING UNIT

Consider two-level Paging unit using a 32-bit linear address divided into three fields - Page directory field containing most significant 10 bits, Page Table field containing intermediate 10 bits and page offset field containing least significant 12 bits.

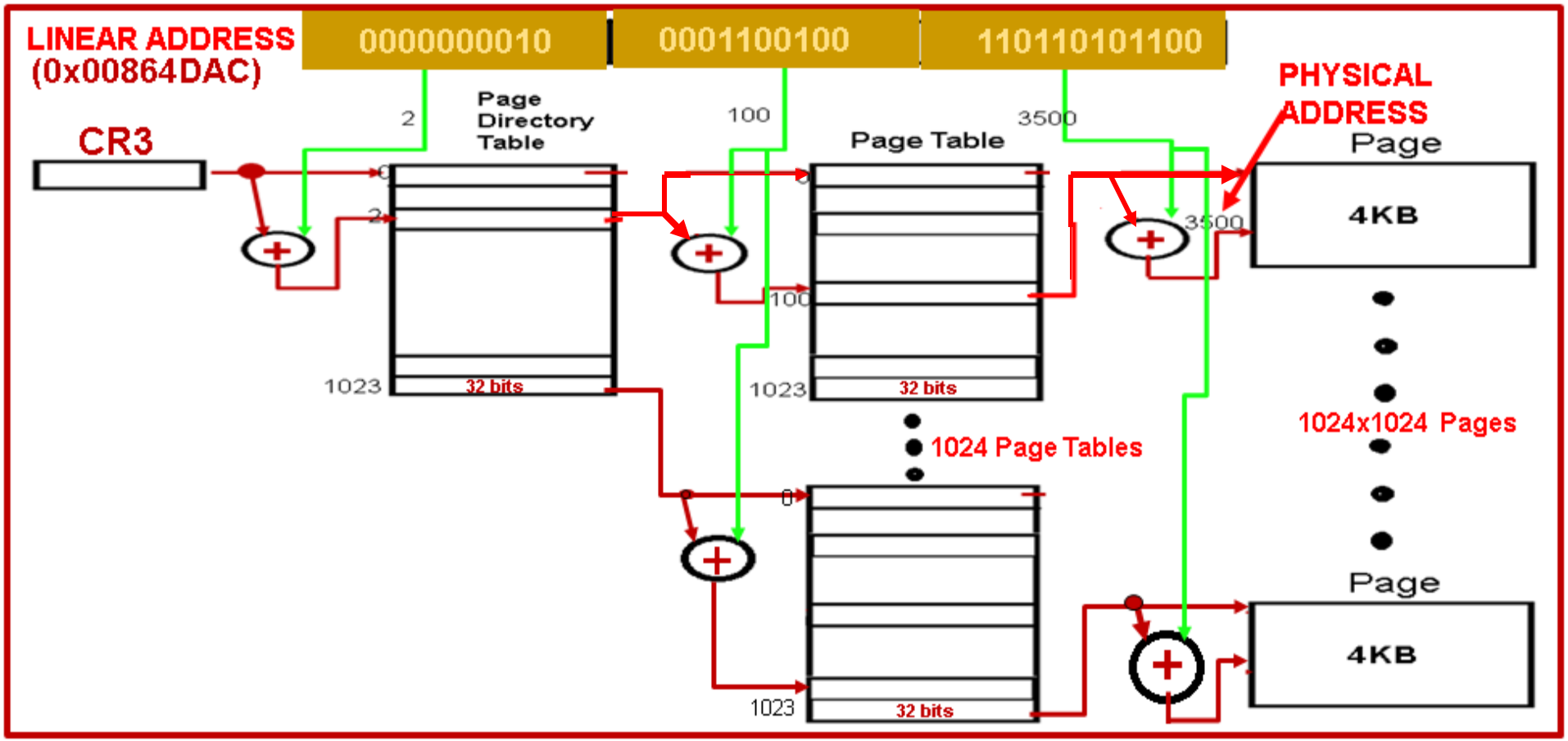


Total number of entries in Page directory Table : 2^{10} = 1024
Total number of entries in each Page Table : 2^{10} = 1024
Total number of Page tables that can be stored : 2^{10} = 1024
Size of each page : 2^{12} = 4KB
Total number of pages that can be stored : 2^{10} x 2^{10} = 2^{20} = 1M
Total amount of main memory required to store these pages : 2^{10} x 2^{10} x 2^{12} = 2^{32} = 4GB

Paging in Hardware

TWO-LEVEL PAGING UNIT

Consider two-level Paging unit using a 32-bit linear address divided into three fields - Page directory field containing most significant 10 bits, Page Table field containing intermediate 10 bits and page offset field containing least significant 12 bits.



(ii) Determine the maximum amount of memory required to store all pages, Page directory & Page tables assuming that each entry in the page directory and page tables take 32-bits (each entry size =32-bits).

Memory required to store 1 Page Directory Table = 2^{10} x 4 bytes = 4KB

Memory required to store 1024 Page Tables = 2^{10} x 2^{10} x 4 bytes=4MB

Memory required to store all Pages = 2^{10} x 2^{10} X 4KB = 4GB

Maximum amount of memory required = 4KB + 4MB + 4GB

Paging in Hardware

- The entries of Page Directories and Page Tables have the same structure.
- Each entry includes the following fields:
- Present flag

If it is set, the referred page (or Page Table) is contained in main memory; if the flag is 0, the page is not contained in main memory and the remaining entry bits may be used by the operating system for its own purposes.

If the entry of a Page Table or Page Directory needed to perform an address translation has the Present flag cleared, the paging unit stores the linear address in the cr2 processor register and generates the exception 14, that is, the "Page fault" exception.

- Field containing the 20 most significant bits of a page frame physical address
  - Since each page frame has a 4 KB capacity, its physical address must be a multiple of 4096, so the 12 least significant bits of the physical address are always equal to 0. If the field refers to a Page Directory, the page frame contains a Page Table; if it refers to a Page Table, the page frame contains a page of data.

Paging in Hardware

- **Accessed flag**
  - Is set each time the paging unit addresses the corresponding page frame. This flag may be used by the operating system when selecting pages to be swapped out. The paging unit never resets this flag; this must be done by the operating system.
- **Dirty flag**
  - Applies only to the Page Table entries. It is set each time a write operation is performed on the page frame. As in the previous case, this flag may be used by the operating system when selecting pages to be swapped out. The paging unit never resets this flag; this must be done by the operating system.
- **Read/Write flag**
  - Contains the access right (Read/Write or Read) of the page or of the Page Table..
- **User/Supervisor flag**
  - Contains the privilege level required to access the page or Page Table
- **Two flags called PCD and PWT**
  - Control the way the page or Page Table is handled by the hardware cache
- **Page Size flag**
  - Applies only to Page Directory entries. If it is set, the entry refers to a 2 MB or a 4 MB long page frame
- **Global flag**

–Applies only to Page Table entries. This flag was introduced in the Pentium Pro to prevent frequently used pages from being flushed from the TLB cache . It works only if the Page Global Enable (PGE) flag of register cr4 is set.

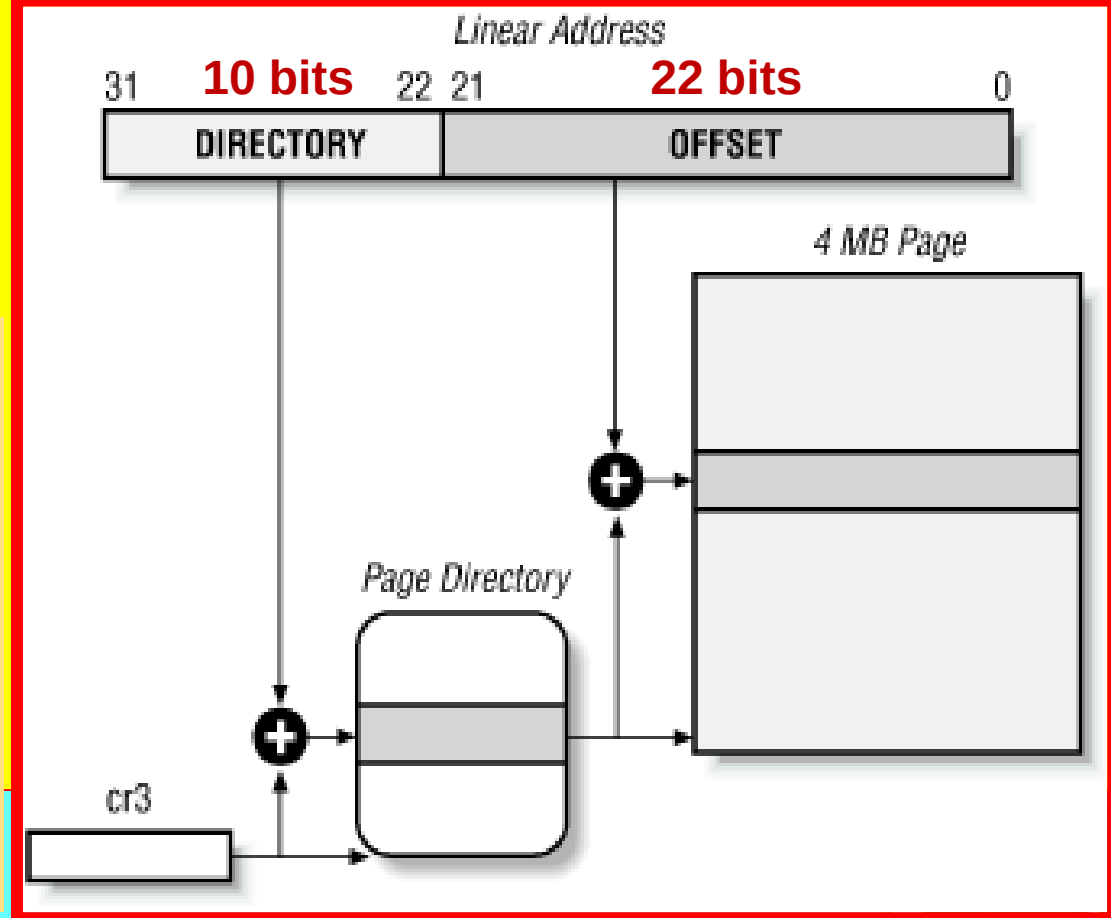
Extended Paging

Extended Paging

- Starting with the Pentium model, Intel 80x86 microprocessors introduce extended paging , which allows page frames to be either 4 KB or 4 MB in size
- Extended paging is enabled by setting the Page Size flag of a Page Directory entry.
- In this case, the paging unit divides the 32 bits of a linear address into two fields:
  - Directory
    - The most significant 10 bits
  - Offset
    - The remaining 22 bits

Page Directory entries for extended paging are the same as for normal paging, except that:

- The Page Size flag must be set.
- Only the first 10 most significant bits of the 20-bit physical address field are significant. This is because each physical address is aligned on a 4 MB boundary, so the 22 least significant bits of the address are 0.



Extended Paging

- Extended paging coexists with regular paging; it is enabled by setting the PSE flag of the cr4 processor register.
- Extended paging is used to translate large intervals of contiguous linear addresses into corresponding physical ones; in these cases, the kernel can do without intermediate Page Tables and thus save memory.

Consider Extended Paging unit using a 32-bit linear address divided into two fields - Page directory field containing most significant 10 bits, and page offset field containing least significant 22 bits.

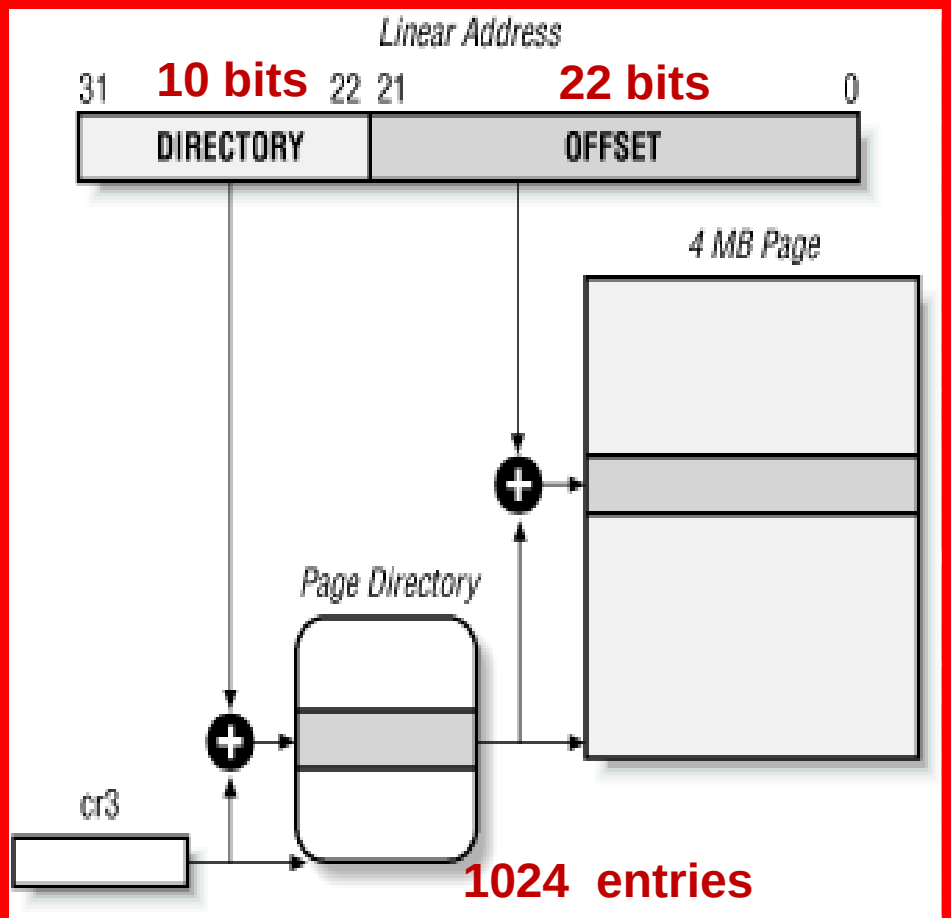
Determine the following :

Total number of entries in Page directory Table = 2^{10} = 1024

Size of each page = 2^{22} = 4MB

Total number of pages that can be stored = 2^{10} = 1024

Total amount of main memory required to store these pages = 2^{10}x 2^{22} = 2^{32} = 4GB



Paging in Hardware

Hardware Protection Scheme

- The paging unit uses a different protection scheme from the segmentation unit.
- While Intel processors allow four possible privilege levels to a segment, only two privilege levels are associated with pages and Page Tables, because privileges are controlled by the User/Supervisor flag
- When User/Supervisor flag is 0, the page can be addressed only when the CPL is less than 3 (this means, for Linux, when the processor is in Kernel Mode).
- When this flag is 1, the page can always be addressed.
- Furthermore, instead of the three types of access rights (Read, Write, Execute) associated with segments, only two types of access rights (Read, Write) are associated with pages.
- If the Read/Write flag of a Page Directory or Page Table entry is equal to 0, the corresponding Page Table or page can only be read; otherwise it can be read and written.

Paging in Hardware

An Example of Paging

- Let us assume that the kernel has assigned the linear address space between 0x20000000 and 0x2003ffff to a running process.
- This space consists of exactly 64 pages.
- Let us start with the 10 most significant bits of the linear addresses assigned to the process, which are interpreted as the Directory field by the paging unit.
- The addresses start with a 2 followed by zeros, so the 10 bits all have the same value, namely 0x080 or 128 decimal.
- Thus the Directory field in all the addresses refers to the 129th entry of the process Page Directory.
- The corresponding entry must contain the physical address of the Page Table assigned to the process.
- If no other linear addresses are assigned to the process, all the remaining 1023 entries of the Page Directory are filled with zeros.
- The values assumed by the intermediate 10 bits, (that is, the values of the Table field) range from ox000 to 0x03f, or from 0 to 63 decimal.
- Thus, only the first 64 entries of the Page Table are significant. The remaining 960 entries are filled with zeros.

0x20000000

0x2003FFFF

LINEAR SEGMENT

Determine the number of pages:
Ending Address : 0x2003FFFF -
Starting Address: 0x20000000
Offset : 0x3FFFF = 2^{18}
Total Size of linear Segment : 2^{18} = 256KB
Total number of Pages = 256KB/4KB = 64 Pages

0x20000000 0010 0000 0000 0000 0000 0000 0000 0000

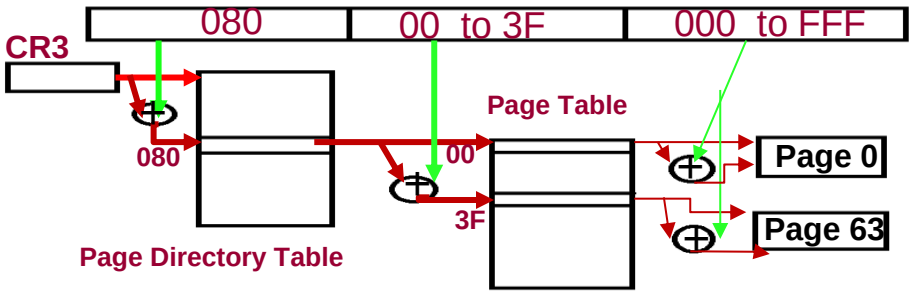
0x2003FFFF 0010 0000 0000 0011 1111 1111 1111 1111

080 — Page Directory field

00 to 3F — Page Table field

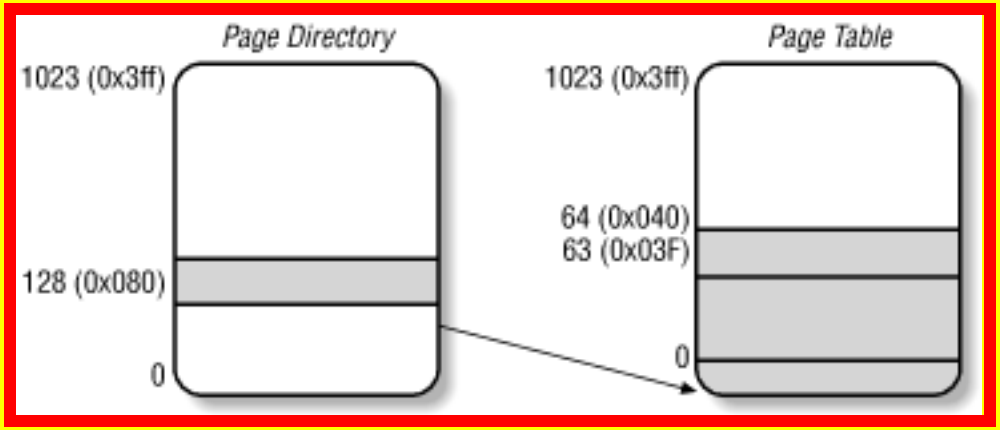
000 to FFF — Page Offset

For all combinations of the linear addresses :
Page directory field contains a fixed value 0x080
Only one entry is required in Page Directory Table, i.e., only one entry at offset ox080
That means only one Page Table is required which is pointed to by the entry at 0x080
Page Table field value range from 0x000 to 0x03F, that is, number of entries in Page table are from offset 00 to 0x3F = 2^6 = 64 entries are required in Page Table each entry pointing to one Page



Paging in Hardware

- **An Example of Paging**



0x20021406

0010000000 0000100001 010000000110

0x080	0x021	0x406
Page Directory entry	Page Table entry	Page Offset

Suppose that the process needs to read the byte at linear address 0x20021406.

This address is handled by the paging unit as follows:

1. The Directory field 0x80 is used to select entry 0x80 of the Page Directory, which points to the Page Table associated with the process's pages.

2. The Table field 0x21 is used to select entry 0x21 of the Page Table, which points to the page frame containing the desired page.

3. Finally, the Offset field 0x406 is used to select the byte at offset 0x406 in the desired page frame.

- **If the Present flag of the 0x21 entry of the Page Table is cleared, the page is not present in main memory; in this case, the paging unit issues a page exception while translating the linear address.**
- **The same exception is issued whenever the process attempts to access linear addresses outside of the interval delimited by 0x20000000 and 0x2003ffff since the Page Table entries not assigned to the process are filled with zeros; in particular, their Present flags are all cleared.**

Memory Addressing

- **Physical Address Extension (PAE) Paging Mechanism**

- **Starting with the Pentium Pro, all Intel processors are now able to address up to 2^{36} = 64 GB of RAM.**
- **However, the increased range of physical addresses can be exploited only by introducing a new paging mechanism that translates 32-bit linear addresses into 36-bit physical ones.**
- **With the Pentium Pro processor, Intel introduced a mechanism called Physical Address Extension (PAE).**
- **PAE is activated by setting the Physical Address Extension (PAE) flag in the cr4 control register.**
- **The Page Size (PS) flag in the page directory entry enables large page sizes (2 MB when PAE is enabled).**

Paging in Hardware

Physical Address Extension (PAE) Paging Mechanism

- Intel has changed the paging mechanism in order to support PAE.
- The 64 GB of RAM are split into 2^{24} distinct page frames, and the physical address field of Page Table entries has been expanded from 20 to 24 bits.
- Because a PAE Page Table entry must include the 12 flag bits and the 24 physical address bits, for a grand total of 36, the Page Table entry size has been doubled from 32 bits to 64 bits.
- As a result, a 4-KB PAE Page Table includes 512 entries instead of 1,024.
- A new level of Page Table called the Page Directory Pointer Table (PDPT) consisting of four 64-bit entries has been introduced.
- The cr3 control register contains a 27-bit Page Directory Pointer Table base address field.
- Because PDPTs are stored in the first 4 GB of RAM and aligned to a multiple of 32 bytes (25), 27 bits are sufficient to represent the base address of such tables.

Paging in Hardware

Physical Address Extension (PAE) Paging Mechanism

- When mapping linear addresses to 4 KB pages (PS flag cleared in Page Directory entry), the 32 bits of a linear address are interpreted in the following way:
  - cr3 -Points to a PDPT (Page Directory Pointer Table)
  - bits 31 to 30 - Point to 1 of 4 possible entries in PDPT
  - bits 29 to 21 - Point to 1 of 512 possible entries in Page Directory
  - bits 20 to 12 - Point to 1 of 512 possible entries in Page Table
  - bits 11 to 0 Offset of 4-KB page
- When mapping linear addresses to 2-MB pages (PS flag set in Page Directory entry), the 32 bits of a linear address are interpreted in the following way:
  - cr3 - Points to a PDPT (Page Directory Pointer Table)
  - bits 31 to 30 - Point to 1 of 4 possible entries in PDPT
  - bits 29 to 21 - Point to 1 of 512 possible entries in Page Directory
  - bits 20 to 0 - Offset of 2-MB page
- To summarize, once cr3 is set, it is possible to address up to 4 GB of RAM
- PAE allows the kernel to exploit up to 64 GB of RAM, and thus to increase significantly the number of processes in the system.

Paging in Hardware

Three-Level Paging

- Two-level paging is used by 32-bit microprocessors.
- But in recent years, several microprocessors (such as Compaq's Alpha, and Sun's UltraSPARC) have adopted a 64-bit architecture.
- In this case, two-level paging is no longer suitable and it is necessary to move up to three-level paging.
- Start by assuming about as large a page size as is reasonable (since you have to account for pages being transferred routinely to and from disk).
- Let's choose 16 KB for the page size.
- Since 1 KB covers a range of 2^{10} addresses,
- 16 KB covers 2^{14} addresses, so the Offset field would be 14 bits.
- This leaves 50 bits of the linear address to be distributed between the Table and the Directory fields.
- If we now decide to reserve 25 bits for each of these two fields, this means that both the Page Directory and the Page Tables of a process would include 2^{25} entries, that is, more than 32 million entries.

Paging in Hardware

Three-Level Paging

- **Even if RAM is getting cheaper and cheaper, we cannot afford to waste large memory space just for storing the page tables.**

The solution chosen for Compaq's Alpha microprocessors is the following:

- **Page frames are 8 KB long, so the Offset field is 13 bits long.**
- **Only the least significant 43 bits of an address(64-bit) are used.**
  (The most significant 21 bits are always set 0.)
- **Three levels of page tables are introduced so that the remaining 30 bits of the address can be split into three 10-bit fields.**

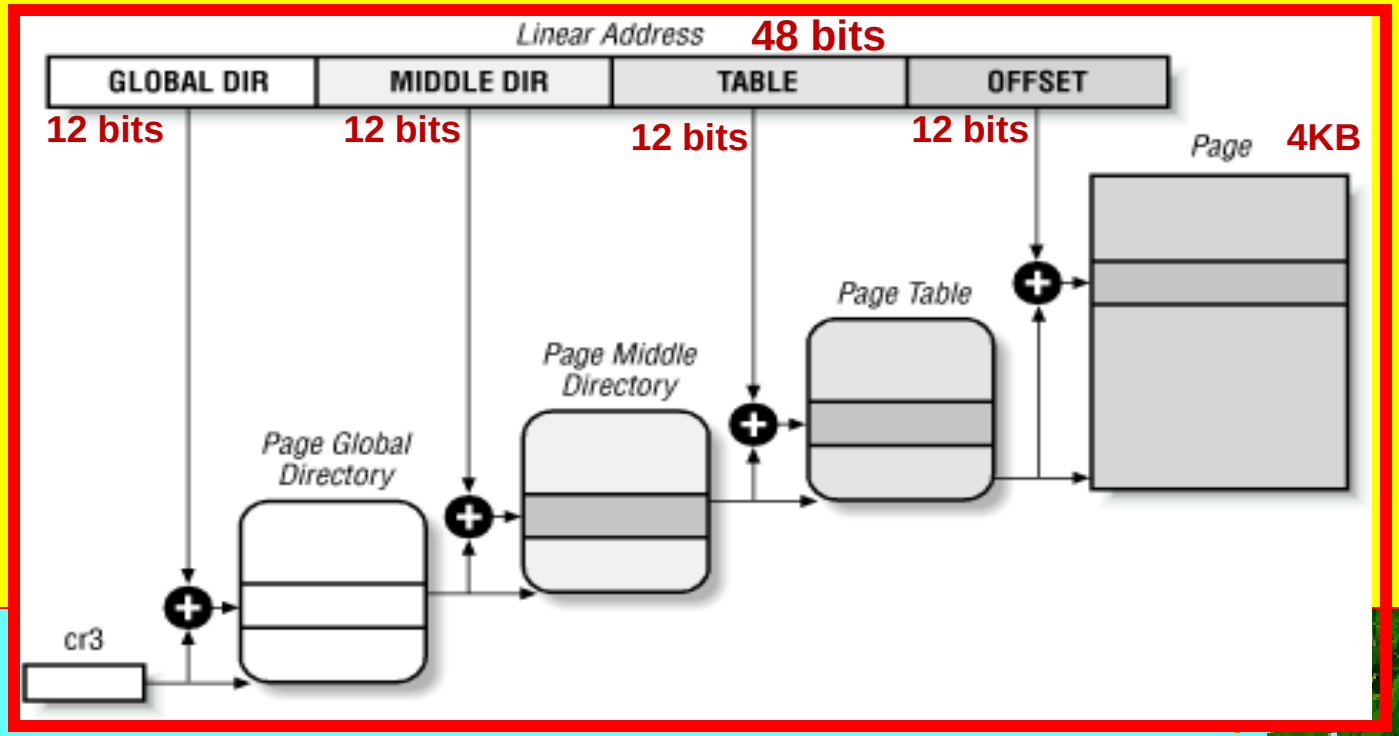
So the Page Tables include 2^{10} = 1024 entries as in the two-level paging schema

- **Linux's designers decided to implement a paging model inspired by the Alpha architecture.**

Paging in Linux

- **Paging in Linux – THREE LEVEL PAGING UNIT**

- **Linux adopts a common paging model that fits both 32-bit and 64-bit architectures**
- **Upto version 2.6.10 Linux adopted a three-level paging model so paging is feasible on 64-bit architectures. This model uses three types of paging tables:**
  - **Page Global Directory**
  - **Page Middle Directory**
  - **Page Table**
- **The Page Global Directory includes the addresses of several Page Middle Directories, which in turn include the addresses of several Page Tables.**
- **Each Page Table entry points to a page frame. The linear address is thus split into four parts.**



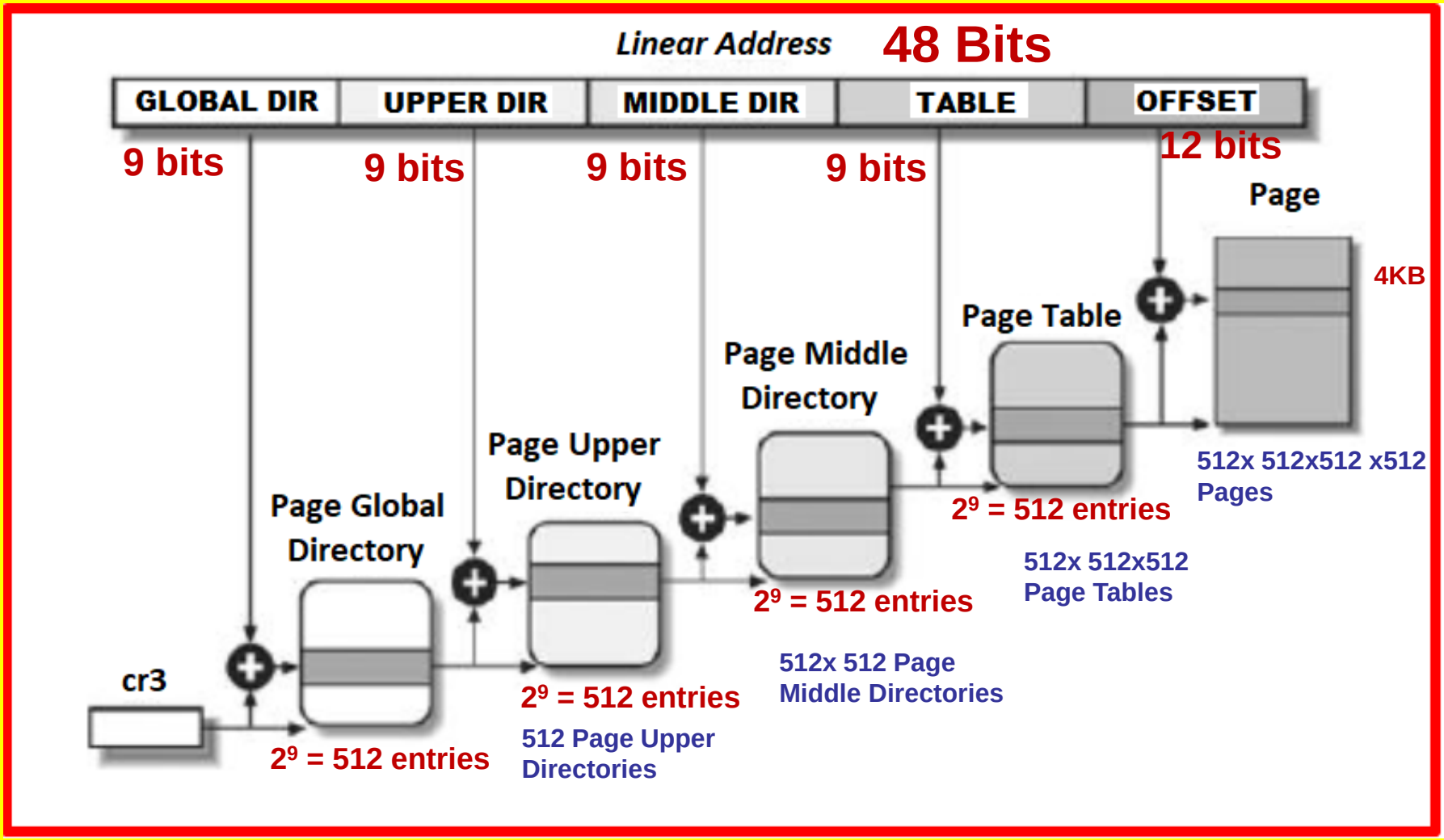
Paging in Linux

- **Paging in Linux - Four-level paging model**

- **Starting with version 2.6.11, a four-level paging model has been adopted.**
- **This change has been made to fully support the linear address bit splitting used by the x86_64 platform**
  - **Page Global Directory**
  - **Page Upper Directory**
  - **Page Middle Directory**
  - **Page Table**
- **The Page Global Directory includes the addresses of several Page Upper Directories, which in turn include the addresses of several Page Middle Directories, which in turn include the addresses of several Page Tables.**
- **Each Page Table entry points to a page frame. Thus the linear address can be split into up to five parts**

Linux Paging Model

- Linux Paging Model



Linux Paging Model

- **Paging in Linux**

- **For 32-bit architectures with no Physical Address Extension, two paging levels are sufficient.**
- **Linux essentially eliminates the Page Upper Directory and the Page Middle Directory fields by saying that they contain zero bits.**
- **However, the positions of the Page Upper Directory and the Page Middle Directory in the sequence of pointers are kept so that the same code can work on 32-bit and 64-bit architectures.**
- **The kernel keeps a position for the Page Upper Directory and the Page Middle Directory by setting the number of entries in them to 1 and mapping these two entries into the proper entry of the Page Global Directory.**
- **For 32-bit architectures with the Physical Address Extension enabled, three paging levels are used.**
- **The Linux's Page Global Directory corresponds to the 80x86's Page Directory Pointer Table, the Page Upper Directory is eliminated, the Page Middle Directory corresponds to the 80 x 86's Page Directory, and the Linux's Page Table corresponds to the 80 x 86's Page Table.**
- **Finally, for 64-bit architectures three or four levels of paging are used depending on the linear address bit splitting performed by the hardware**

Paging in Linux

- Linux handling of processes relies heavily on paging.
- In fact, the automatic translation of linear addresses into physical ones makes the following design objectives feasible:
  - Assign a different physical address space to each process, thus ensuring an efficient protection against addressing errors.
  - Distinguish pages, that is, groups of data, from page frames, that is, physical addresses in main memory.

    This allows the same page to be stored in a page frame, then saved to disk, and later reloaded in a different page frame.

    This is the basic ingredient of the virtual memory mechanism
- Each process has its own Page Global Directory and its own set of Page Tables.
- When a process switching occurs, Linux saves in a TSS segment the contents of the cr3 control register and loads from another TSS segment a new value into cr3.
- Thus, when the new process resumes its execution on the CPU, the paging unit refers to the correct set of page tables.

Linux Paging Model

- What happens when this three-level paging model is applied to the Pentium, which uses only two types of page tables?

- Linux essentially eliminates the Page Middle Directory field by saying that it contains zero bits.
- However, the position of the Page Middle Directory in the sequence of pointers is kept so that the same code can work on 32-bit and 64-bit architectures.

- The kernel keeps a position for the Page Middle Directory by setting the number of entries in it to 1 and mapping this single entry into the proper entry of the Page Global Directory.
- Mapping logical to linear addresses now becomes a mechanical task, although somewhat complex.

Paging for 64-bit Architectures

Paging for 64-bit Architectures

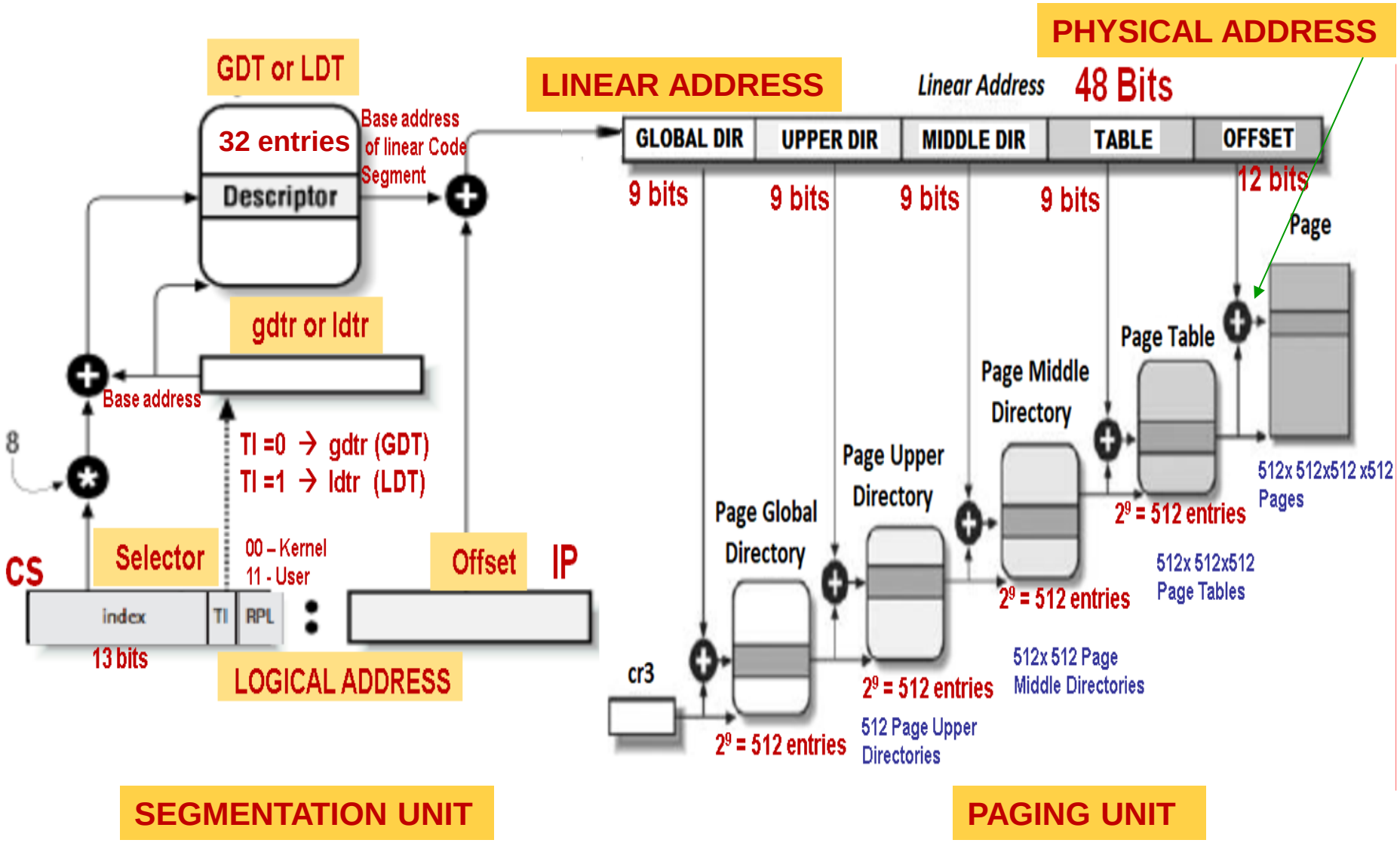
- **Table summarizes the main characteristics of the hardware paging systems used by some 64-bit platforms supported by Linux.**

Paging levels in some 64-bit architectures

Platform Name	Page Size	Number of address bits used	Number of Paging Levels	Linear Address Splitting
ALPHA DEC	8KB	43	3	10 + 10 + 10 + 13
IA64 Intel	4KB	39	3	9 + 9+ 9 + 12
PPC64 PowerPC	4KB	41	3	10 + 10 + 9 + 12
SH64 SuperH Hitachi	4KB	41	3	10 + 10 + 9 + 12
X86_64 Intel, AMD	**4KB**	**48**	**4**	**9 + 9 + 9 + 9 + 12**

LINUX - SEGMENTATION & PAGING UNITS

- ADDRESS TRANSLATION FROM LOGICAL ADDRESS TO LINEAR ADDRESS & LINEAR ADDRESS TO PHYSICAL ADDRESS



MEMORY ADDRESSING - SEGMENTATION UNIT

Questions

- Explain what is meant by paging.
- Describe the function of a Paging unit.
- Distinguish between page and page frame.
- The data structures that map linear addresses to physical addresses are called ……..
- In Intel processors, paging is enabled by setting the ……. Flag in the …….. Register.

  a) PG , cr4 b) PAE, cr4 c) PG, cr0 d) PAE, cr0
- In paging the user provides only _____, which is partitioned by the hardware into _____ and _____.

  a) one page offset, page number, address

  b) one address, page number, page offset

  c) one page number, page offset, address

  d) None of these
- Explain the purpose of the following flags in each of the Page Table entries:

  a) Present b) Accessed c) Dirty d) Read/Write

  e) User/Supervisor f) PCD & PWT g) Page Size f) Global
- Run time mapping from linear to physical address is done by

  a) memory management unit b) CPU c) PCI

  d) none of these
- The address of a page directory table in memory is pointed by

  a) stack pointer b) control register c) page pointer d) program counter

MEMORY ADDRESSING - SEGMENTATION UNIT

Questions

- CR3 register contains the base address of ……
- In Intel processors, paging is enabled by setting the ……. flag of the …….register
- As far as processor is concerned, Program always deals with
  a) logical address b) absolute address
  c) physical address d) relative address
- The page table contains
  a) base address of each page in physical memory
  b) page offset
  c) page size
  d) none of the mentioned
- Operating System maintains the page table for
  a) each process b) each thread
  c) each instruction d) each address
- Physical memory is broken into fixed-sized blocks called _____.
  a) Page frames b) Pages c) Segments d) Directories
- The size of a page is typically :
  a) varied b) power of 2 c) power of 4 d) None of these
- The operating system maintains a _____ table that keeps track of how many frames have been allocated, how many are there, and how many are available.
  a) page b) mapping c) frame d) memory
- Draw the Two-level Paging Unit showing clearly all the blocks used to map a 32-bit linear address to 32-bit physical address using a regular page size of 4KB.

MEMORY ADDRESSING - SEGMENTATION UNIT

Questions

- The ……..flag in the page directory entry enables large page sizes (4 MB when PSE is enabled
- Extended paging coexists with regular paging and is enabled by setting the ……flag of the …….. processor register
- The physical address of the Page Directory in use is stored in processor control register .....

  a. cr0 b. cr1 c. cr2 d. cr3
- Consider two-level Paging unit using a 32-bit linear address divided into three fields - Page directory containing most significant 10 bits, Page Table containing intermediate 10 bits and page offset containing least significant 12 bits.

  (i) Draw the Paging unit showing all the Tables and their corresponding entries.

  (ii) Determine the number of entries in Page directory (i.e., maximum number of Page tables that can be stored)

  (iii) Determine the number entries in each Page Table (i.e., maximum number of pages that can be stored)

  (iv) Determine the total number of Page tables that can be stored

  (v) Determine the size of each page

  (vi) Determine the total number of pages that can be stored and total amount of main memory required to store these pages

  (vii) Determine the maximum amount of memory required to store all pages, Page directory & Page tables assuming that each entry in the page directory and page tables take 32-bits (each entry size =32-bits).
- What is meant by Extended Paging. Explain Extended Paging with the help of a suitable diagram by taking a 32-bit linear address.
- The ……..flag in the page directory entry enables large page sizes (2 MB when PAE is enabled

MEMORY ADDRESSING - SEGMENTATION UNIT

Questions

- Draw the Two-level Paging Unit showing clearly all the blocks used to map a 32-bit linear address to 32-bit physical address using an extended page size of 4MB.
- Explain three-level Paging used in linux systems using a suitable diagram by taking a 64-bit linear address
- Explain four-level Paging used in linux systems using a suitable diagram considering a 64-bit linear address.
- Consider four-level Paging unit using a 48-bit linear address divided into five fields - Global Page directory containing most significant 9 bits, Upper Page directory containing next 9 bits, Middle Page directory containing next 9 bits, Page Table containing next 9 bits and page offset containing least significant 12 bits.

  (i) Draw the Paging unit showing all the Tables and their corresponding entries.

  (ii) Determine the number of entries in Global Page directory Table, Upper Page Directory Table, Middle Page Directory Table, and Page Table.

  (iii) Determine Total number of Page Tables (i.e., maximum number of Page tables that can be stored)

  (iv) Determine the number entries in each Page Table (i.e., maximum number of pages that can be stored)

  (v) Determine the size of each page

  (vii) Determine the Total number of pages that can be stored and total amount of memory required to store these pages.

  (vii) Determine the maximum amount of memory required to store all pages, Page directory tables & Page tables assuming that each entry in the page directory tables and page tables take 64-bits (each entry size =64bits).
- Extended Paging is enabled by setting the ……. Flag of the …… processor control register

  a) PAE , cr4 b) PSE, cr4 c) PAE, cr0 d) PSE, cr0
- Explain what is meant by Page Fault. Indicate which flag in the Page table entry is checked to raise Page fault exception.

MEMORY ADDRESSING - SEGMENTATION UNIT

Questions

- Assume that the kernel has assigned the linear address space between 0x20000000 and 0x200fffff to a running process. Two-level Paging is used with page size 4K.

  Determine the number of pages allocated in the segment for the process.

  Determine the number of entries used in the Page Directory Table and Page Table.

- Suppose that the process needs to read the byte at linear address 0x20042646 then determine which entries in the Page Directory table, Page Table and Page frame in the memory are accessed.

- Explain what is meant by Physical Address Extension (PAE) Paging mechanism.

- In a system using Intel PAE paging mechanism, explain using a block diagram how linear address (41-bit with PAE enabled and Page Size 2 MB) is translated into physical address clearly showing PDPT, Page Directory Table, Page Table and Page frame.

- What happens when the three-level paging model is applied to the Pentium, which uses only two types of page tables?

MEMORY ADDRESSING - SEGMENTATION UNIT

Questions

- Consider a system having Intel PAE paging mechanism using a 41-bit linear address (with PAE enabled and Page Size 2MB) divided into four fields - Page Directory Pointer Table PDPT(2 bits), Page Directory Table(9 bits), Page Table(9 bits) and Page frame offset (21 bits).

  (i) Draw the Paging unit showing all the Tables and their corresponding entries.

  (ii) Determine the number of entries in Page Directory Pointer Table PDPT, Page Directory Table, and Page Table.

  (iii) Determine Total number of Page Directory Tables and Page Tables (i.e., maximum number of Page tables that can be stored)

  (iv) Determine the number entries in each Page Table (i.e., maximum number of pages that can be stored)

  (v) Determine the Total number of pages that can be stored and total amount of memory required to store these pages.

  (vi) Determine the maximum amount of memory required to store all pages, Page Directory Pointer Table, Page directory tables & Page tables assuming that each entry in the Page Directory Pointer Table, page directory tables and page tables take 64-bits (each entry size =64bits).

- Explain the Paging model used in Linux.
- Describe the entire process of translating the logical address to physical address by drawing the complete block diagram consisting of Segmentation Unit and Paging Unit showing all the components of Segmentation Unit (cs, ip, gdtr, GDT, etc) and Paging Unit (Two level paging with cr3, Page Directory table, Page Tables, Pages,.. etc) and their interconnections. Also show the computation of physical addresses of entries of all the tables used in both Segmentation and Paging units.

SOC 3010
OPERATING SYSTEM

READING ASSIGNMENT

MEMORY ADDRESSING

PAGING UNIT

CACHE, TLB, PHYSICAL MEMORY LAYOUT

Processor hardware cache

Hardware Cache

- Today's microprocessors have clock rates approaching gigahertz, while dynamic RAM (DRAM) chips have access times in the range of tens of clock cycles.
- This means that the CPU may be held back considerably while executing instructions that require fetching operands from RAM and/or storing results into RAM.
- Hardware cache memories have been introduced to reduce the speed mismatch between CPU and RAM.
- They are based on the well-known locality principle, which holds both for programs and data structures: because of the cyclic structure of programs and the packing of related data into linear arrays, addresses close to the ones most recently used have a high probability of being used in the near future.
- It thus makes sense to introduce a smaller and faster memory that contains the most recently used code and data.

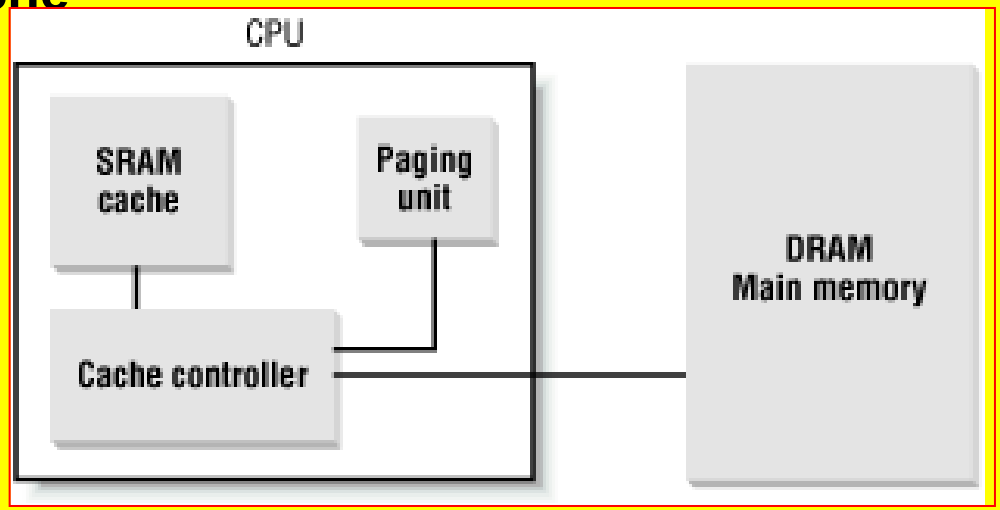
Processor hardware cache

- **Hardware Cache**

- **For this purpose, a new unit called the line has been introduced into the Intel architecture.**
- **It consists of a few dozen contiguous bytes that are transferred in burst mode between the slow DRAM and the fast onchip static RAM (SRAM) used to implement caches.**
- **The cache is subdivided into subsets of lines.**
- **At one extreme the cache can be direct mapped, in which case a line in main memory is always stored at the exact same location in the cache.**
- **At the other extreme, the cache is fully associative, meaning that any line in memory can be stored at any location in the cache.**
- **But most caches are to some degree N-way associative, where any line of main memory can be stored in any one of N lines of the cache.**
- **For instance, a line of memory can be stored in two different lines of a 2-way set of associative cache.**

Processor hardware cache

- **Hardware Cache**



- **As shown in Figure, the cache unit is inserted between the paging unit and the main memory.**
- **It includes both a hardware cache memory and a cache controller.**
- **The cache memory stores the actual lines of memory.**
- **The cache controller stores an array of entries, one entry for each line of the cache memory. Each entry includes a tag and a few flags that describe the status of the cache line.**
- **The tag consists of some bits that allow the cache controller to recognize the memory location currently mapped by the line.**
- **The bits of the memory physical address are usually split into three groups: the most significant ones correspond to the tag, the middle ones correspond to the cache controller subset index, the least significant ones to the offset within the line.**

Processor hardware cache

Processor hardware cache

- When accessing a RAM memory cell, the CPU extracts the subset index from the physical address and compares the tags of all lines in the subset with the high-order bits of the physical address.
- If a line with the same tag as the high-order bits of the address is found, the CPU has a cache hit; otherwise, it has a cache miss.
- When a cache hit occurs, the cache controller behaves differently depending on access type.
- For a read operation, the controller selects the data from the cache line and transfers it into a CPU register; the RAM is not accessed and the CPU achieves the time saving for which the cache system was invented.
- For a write operation, the controller may implement one of two basic strategies called write-through and write-back.
- In a write-through, the controller always writes into both RAM and the cache line, effectively switching off the cache for write operations.
- In a write-back, which offers more immediate efficiency, only the cache line is updated, and the contents of the RAM are left unchanged.
- After a write-back, of course, the RAM must eventually be updated.

Processor hardware cache

- **Processor hardware cache**

- The cache controller writes the cache line back into RAM only when the CPU executes an instruction requiring a flush of cache entries or when a FLUSH hardware signal occurs (usually after a cache miss).
- When a cache miss occurs, the cache line is written to memory, if necessary, and the correct line is fetched from RAM into the cache entry.
- Multiprocessor systems have a separate hardware cache for every processor, and therefore they need additional hardware circuitry to synchronize the cache contents.
- Cache technology is rapidly evolving.
- For example, the first Pentium models included a single on-chip cache called the L1-cache.
- More recent models also include other larger and slower on-chip caches called the L2-cache, L3-cache
- The consistency between the cache levels is implemented at the hardware level.
- Linux ignores these hardware details and assumes there is a single cache.

Processor hardware cache

- **Processor hardware cache**
- **The CD flag of the cr0 processor register is used to enable or disable the cache circuitry.**
- **The NW flag, in the same register, specifies whether the write-through or the write-back strategy is used for the caches.**
- **Another interesting feature of the Pentium cache is that it lets an operating system associate a different cache management policy with each page frame.**
- **For that purpose, each Page Directory and each Page Table entry includes two flags: PCD, PWT**
- **PCD specifies whether the cache must be enabled or disabled while accessing data included in the page frame;**
- **PWT specifies whether the write-back or the write-through strategy must be applied while writing data into the page frame.**
- **Linux clears the PCD and PWT flags of all Page Directory and Page Table entries: as a result, caching is enabled for all page frames and the write-back strategy is always adopted for writing**
- **The L1_CACHE_BYTES macro yields the size of a cache line on a Pentium, that is, 32 bytes. In order to optimize the cache hit rate, the kernel adopts the following rules:**
  - **The most frequently used fields of a data structure are placed at the low offset within the data structure so that they can be cached in the same line.**
  - **When allocating a large set of data structures, the kernel tries to store each of them in memory so that all cache lines are uniformly used.**

Translation Lookaside Buffers (TLB)

- **Translation Lookaside Buffers (TLB)**

- **Besides general-purpose hardware caches, Intel 80x86 processors include other caches called translation lookaside buffers or TLB to speed up linear address translation.**

- **When a linear address is used for the first time, the corresponding physical address is computed through slow accesses to the page tables in RAM.**

- **The physical address is then stored in a TLB entry, so that further references to the same linear address can be quickly translated.**

- **The invlpg instruction can be used to invalidate (that is, to free) a single entry of a TLB.**

Translation Lookaside Buffers (TLB)

Translation Lookaside Buffers (TLB)

- In order to invalidate all TLB entries, the processor can simply write into the cr3 register that points to the currently used Page Directory.
- Since the TLBs serve as caches of page table contents, whenever a Page Table entry is modified, the kernel must invalidate the corresponding TLB entry.
- To do this, Linux makes use of the flush_tlb_page(addr) function, which invokes __flush_tlb_one( ).
- The latter function executes the invlpg Assembly instruction:

```
movl $addr,%eax
invlpg (%eax)
```

- Sometimes it is necessary to invalidate all TLB entries, such as during kernel initialization.
- In such cases, the kernel invokes the __flush_tlb( ) function, which rewrites the current value of cr3 back into it:

```
movl %cr3, %eax
.....
movl %eax, %cr3
```

Physical Memory Layout

Reserved Page Frames

- During the initialization phase the kernel must build a physical addresses map that specifies which physical address ranges are usable by the kernel and which are unavailable (either because they map hardware devices' I/O shared memory or because the corresponding page frames contain BIOS data).
- The kernel considers the following page frames as reserved :
  - Those falling in the unavailable physical address ranges
  - Those containing the kernel's code and initialized data structures

  A page contained in a reserved page frame can never be dynamically assigned or swapped to disk.
- As a general rule, the Linux kernel is installed in RAM starting from the physical address 0x00100000 i.e., from the second megabyte.
- The total number of page frames required depends on how the kernel is configured.
- A typical configuration yields a kernel that can be loaded in less than 3 MB of RAM.

Physical Memory Layout

- Why isn't the kernel loaded starting with the first available megabyte of RAM?
- Well, the PC architecture has several peculiarities that must be taken into account:
  - Page frame 0 is used by BIOS to store the system hardware configuration detected during the Power-On Self-Test (POST).
  - Physical addresses ranging from 0x000A0000 to 0x000FFFFF are reserved to BIOS routines and to map the internal memory of ISA graphics cards. This area is the well-known hole from 640 KB to 1 MB in all IBM-compatible PCs: the physical addresses exist but they are reserved, and the corresponding page frames cannot be used by the operating system.
  - Additional page frames within the first megabyte may be reserved by specific computer models. For example, the IBM ThinkPad maps the 0xA0 page frame into the 0x9F one.

Physical Memory Layout

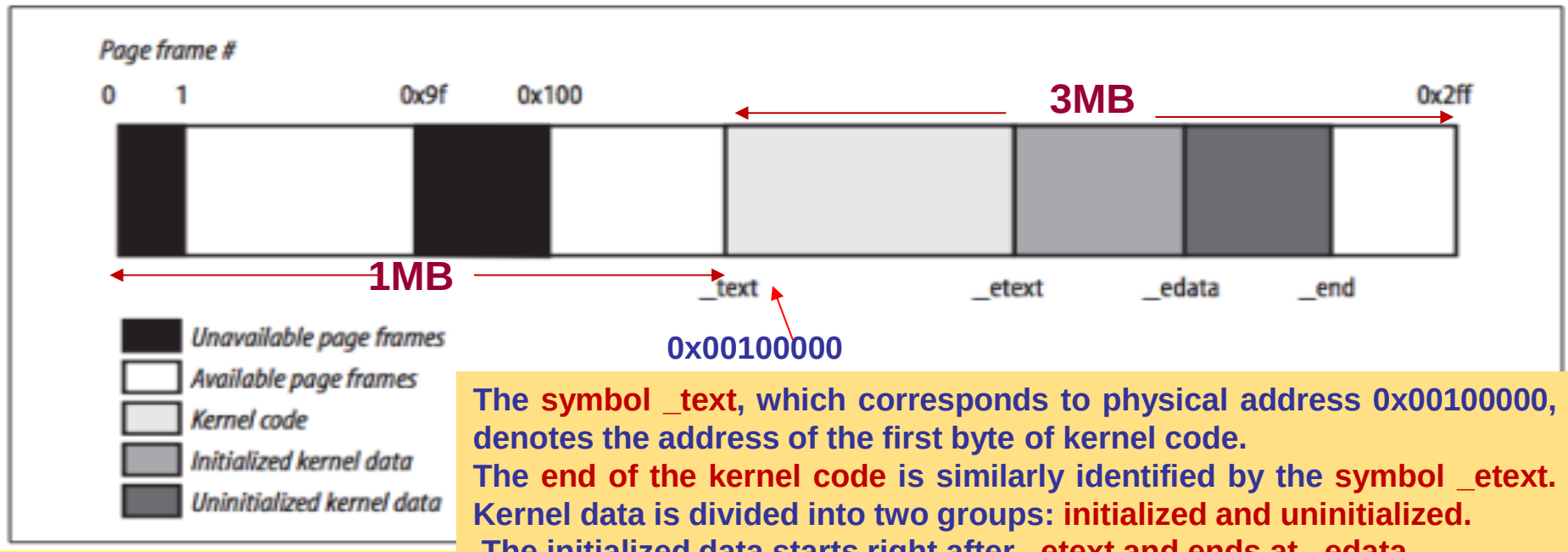
- A typical configuration for a computer having 128 MB of RAM is shown in Table. The physical address range from 0x07FF0000 to 0x07FF2FFF stores information about the hardware devices of the system written by the BIOS in the POST phase.
- During the initialization phase, the kernel copies such information in a suitable kernel data structure, and then considers these page frames usable.
- Conversely, the physical address range of 0x07FF3000 to 0x07FFFFFF is mapped to ROM chips of the hardware devices.
- The physical address range starting from 0xFFFF0000 is marked as reserved, because it is mapped by the hardware to the BIOS's ROM chip

Start	End	Type
0x00000000	0x0009FFFF	Usable
0x000F0000	0x000FFFFF	Reserved
0x00100000	0x07FEFFFF	Usable
0x07FF0000	0x07FF2FFF	ACPI Data
0x07FF3000	0x07FFFFFF	ACPI NVS
0xFFFF0000	0xFFFFFFFF	Reserved

ACPI Non-Volatile Sleeping (NVS) memory region that is allocated and defined so that a system BIOS can save CMOS based memory content at the ACPI NVS memory region during power on system test (POST). The ACPI NVS memory region and it's associated content, is accessible to both OS and non-OS software during runtime execution

Physical Memory Layout

- In order to avoid loading the kernel into groups of noncontiguous page frames, Linux prefers to skip the first megabyte of RAM.
- Clearly, page frames not reserved by the PC architecture will be used by Linux to store dynamically assigned pages.
- Figure shows how the next 3 MB of RAM are filled by Linux assuming that the kernel requires less than 3MB of RAM



The symbol _text, which corresponds to physical address 0x00100000, denotes the address of the first byte of kernel code.
The end of the kernel code is similarly identified by the symbol _etext.
Kernel data is divided into two groups: initialized and uninitialized.
The initialized data starts right after _etext and ends at _edata.
The uninitialized data follows and ends up at _end.

You can find the linear address of these symbols in the file System.map, which is created right after the kernel is compiled. The linear address corresponding to the first physical address reserved to the BIOS or to a hardware device (usually, 0x0009F000) is stored in the i386_endbase variable. In most cases, this variable is initialized with a value written by the BIOS during the POST phase

Process Page Tables

Process Page Tables

- The linear address space of a process is divided into two parts:
  - Linear addresses from 0x00000000 to PAGE_OFFSET -1 can be addressed when the process is in either User or Kernel Mode.
  - Linear addresses from PAGE_OFFSET to 0xFFFFFFFF can be addressed only when the process is in Kernel Mode.
- Usually, the PAGE_OFFSET macro yields the value 0xC0000000: this means that the fourth gigabyte of linear addresses is reserved for the kernel, while the first three gigabytes are accessible from both the kernel and the user programs.
- However, the value of PAGE_OFFSET may be customized by the user when the Linux kernel image is compiled.
- The range of linear addresses reserved for the kernel must include a mapping of all physical RAM installed in the system
- The kernel also makes use of the linear addresses in this range to remap non-contiguous page frames into contiguous linear addresses. Therefore, if Linux must be installed on a machine having a huge amount of RAM, a different arrangement for the linear addresses might be necessary.
- The content of the first entries of the Page Global Directory that map linear addresses lower than PAGE_OFFSET (usually the first 768 entries) depends on the specific process. Conversely, the remaining entries are the same for all processes; they are equal to the corresponding entries of the swapper_ pg_dir kernel Page Global Directory.

Kernel Page Tables

- **Kernel Page Tables**

- **We now describe how the kernel initializes its own page tables.**
- **This is a two-phase activity.**
- **In fact, right after the kernel image has been loaded into memory, the CPU is still running in real mode; thus, paging is not enabled.**
- **In the first phase, the kernel creates a limited 4 MB address space, which is enough for it to install itself in RAM.**
- **In the second phase, the kernel takes advantage of all of the existing RAM and sets up the paging tables properly.**

Kernel Page Tables

❑Provisional kernel page tables

- A provisional Page Global Directory is initialized statically during kernel compilation, while the provisional Page Tables are initialized by the startup_32() assembly language function defined in arch/i386/kernel/head.S.
- The provisional Page Global Directory is contained in the swapper_pg_dir variable.
- The provisional Page Tables are stored starting from pg0, right after the end of the kernel's uninitialized data segments (symbol _end in Figure).
- For the sake of simplicity, let's assume that the kernel's segments, the provisional Page Tables, and the 128 KB memory area fit in the first 8 MB of RAM.
- In order to map 8 MB of RAM, two Page Tables are required.
- The objective of this first phase of paging is to allow these 8 MB of RAM to be easily addressed both in real mode and protected mode.
- Therefore, the kernel must create a mapping from both the linear addresses 0x00000000 through 0x007FFFFF and the linear addresses 0xC0000000 through 0xC07FFFFF into the physical addresses 0x00000000 through 0x007FFFFF.
- In other words, the kernel during its first phase of initialization can address the first 8 MB of RAM by either linear addresses identical to the physical ones or 8 MB worth of linear addresses, starting from 0xC0000000.

Kernel Page Tables

❑Provisional kernel page tables

- The Kernel creates the desired mapping by filling all the swapper_pg_dir entries with zeroes, except for entries 0, 1, 0x300 (decimal 768), and 0x301 (decimal 769); the latter two entries span all linear addresses between 0xC0000000 and 0xC07FFFFF.
- The 0, 1, 0x300, and 0x301 entries are initialized as follows:
  - The address field of entries 0 and 0x300 is set to the physical address of pg0, while the address field of entries 1 and 0x301 is set to the physical address of the page frame following pg0.
  - The Present, Read/Write, and User/Supervisor flags are set in all four entries
  - The Accessed, Dirty, PCD, PWD, and Page Size flags are cleared in all four entries.
  - The startup_32( ) assembly language function also enables the paging unit.
- This is achieved by loading the physical address of swapper_pg_dir into the CR3 control register and by setting the PG flag of the CR0 control register, as shown in the following equivalent code fragment:

```
movl $swapper_pg_dir-0xc0000000,%eax
movl %eax,%cr3       /* set the page table pointer.. */
movl %cr0,%eax
orl $0x80000000,%eax
movl %eax,%cr0       /* ..and set paging (PG) bit */
```

Kernel Page Tables

Final kernel page table

- The final mapping provided by the kernel page tables must transform linear addresses starting from 0xc0000000 into physical addresses starting from 0.
- The __pa macro is used to convert a linear address starting from PAGE_OFFSET to the corresponding physical address, while the __va macro does the reverse.
- The master kernel Page Global Directory is still stored in swapper_pg_dir. It is initialized by the paging_init( ) function, which does the following:
  1. Invokes pagetable_init( ) to set up the Page Table entries properly.
  2. Writes the physical address of swapper_pg_dir in the cr3 control register.
  3. If the CPU supports PAE and if the kernel is compiled with PAE support, sets the PAE flag in the cr4 control register.
  4. Invokes _ _flush_tlb_all( ) to invalidate all TLB entries
- The actions performed by pagetable_init( ) depend on both the amount of RAM present and on the CPU model.

MEMORY ADDRESSING - SEGMENTATION UNIT

Questions

- Describe Hardware Cache organization.
- Explain how memory physical address is used to access a cache line or block
- Describe the two basic strategies used for writing to the cache
- Distinguish between Write-through and write back.
- The …….. flag of the ……. processor control register is used to enable or disable the cache circuitry.

  a) PAE , cr4 b) CD, cr4 c) PAE, cr0 d) CD, cr0
- The …….. flag, of the ……processor control register, specifies whether the write-through or the write-back strategy is used for the caches

  a) PAE , cr4 b) NW, cr4 c) NW, cr0 d) PSE, cr0
- What is TLB? State the purpose of TLB
- Explain what is meant by TLB miss.
- The symbol _text, which corresponds to physical address ……

  a) 0x00100000 b) 0x90000000 c)0xC0000000 d) 0xFFFFFFFF
- PAGE_OFFSET macro refers to the address …….

  a) 0x00100000 b) 0x90000000 c)0xC0000000 d) 0xFFFFFFFF
- Describe how the kernel initializes its own page tables during the booting process.
- State whether the following statement is TRUE or FALSE – “The fourth gigabyte of linear addresses is reserved for the kernel, while the first three gigabytes are accessible from both the kernel and the user programs.”
- A provisional Page Global Directory is initialized statically during kernel compilation, while the provisional Page Tables are initialized by the ……. assembly language fuction
- The master kernel Page Global Directory is initialized by the ……function.
- List the four operations performed by the paging_init() function.

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 2 of Text Book

Text Book

- **Understanding the Linux Kernel**
  - **Daniel P/ Bovet and Marco Cesati**
    Edition, O' Reilly Media 2005, ISBN 978-0596005658

SOC 3010
OPERATING SYSTEMS

MEMORY ADDRESSING

PAGE TABLE HANDLING FUNCTIONS/MACROS

Linear Address Fields

The Linear Address Fields

The following macros simplify page table handling:

PAGE_SHIFT

- **Specifies the length in bits of the Offset field; when applied to Pentium processors it yields the value 12.**
- **Since all the addresses in a page must fit in the Offset field, the size of a page on Intel 80x86 systems is 2^{12} or the familiar 4096 bytes; the PAGE_SHIFT of 12 can thus be considered the logarithm base 2 of the total page size.**
- **This macro is used by PAGE_SIZE to return the size of the page.**
- **Finally, the PAGE_MASK macro is defined as the value 0xfffff000; it is used to mask all the bits of the Offset field.**

Linear Address Fields

- **The Linear Address Fields**

PMD_SHIFT

- **Determines the number of bits in an address that are mapped by the second-level page table.**
- **The PMD_SIZE macro computes the size of the area mapped by a single entry of the Page Middle Directory, that is, of a Page Table.**
- **When PAE is disabled, it yields the value 22 (12 from Offset plus 10 from Table). Thus, PMD_SIZE yields 2^{22} or 4 MB. The PMD_MASK macro yields the value 0xffc00000; it is used to mask all the bits of the Offset and Table fields.**
- **When PAE is enabled, PMD_SHIFT yields the value 21 (12 from Offset plus 9 from Table), PMD_SIZE yields 2^{21} or 2 MB, and PMD_MASK yields 0xffe00000.**

Linear Address Fields

- **The Linear Address Fields**

PUD_SHIFT

- **Determines the logarithm of the size of the area a Page Upper Directory entry can map.**
- **The PUD_SIZE macro computes the size of the area mapped by a single entry of the Page Global Directory.**
- **The PUD_MASK macro is used to mask all the bits of the Offset, Table, Middle Area, and Upper Area fields.**
- **On the 80 x 86 processors, PUD_SHIFT is always equal to PMD_SHIFT and PUD_SIZE is equal to 4 MB or 2 MB.**

Linear Address Fields

- **The Linear Address Fields**

PGDIR_SHIFT

- **Determines the logarithm of the size of the area that a Page Global Directory entry can map.**
- **The PGDIR_SIZE macro computes the size of the area mapped by a single entry of the Page Global Directory.**
- **The PGDIR_MASK macro is used to mask all the bits of the Offset, Table, Middle Area, and Upper Area fields.**

- **When PAE is disabled, PGDIR_SHIFT yields the value 22 (the same value yielded by PMD_SHIFT and by PUD_SHIFT), PGDIR_SIZE yields 2^{22} or 4 MB, and PGDIR_MASK yields 0xffc00000.**
- **Conversely, when PAE is enabled, PGDIR_SHIFT yields the value 30 (12 from Offset plus 9 from Table plus 9 from Middle Area), PGDIR_SIZE yields 2^{30} or 1 GB, and PGDIR_MASK yields 0xc0000000.**

Page Table Handling

- **The kernel also provides several macros and functions to read or modify page table entries:**

- **The pte_none(), pmd_none(), pud_none(), and pgd_none()** macros yield the value 1 if the corresponding entry has the value 0; otherwise, they yield the value 0.

- **The pte_present( ), pmd_present( ), pud_present( ), and pgd_present( )** macros yield the value 1 if the Present flag of the corresponding entry is equal to 1, that is, if the corresponding page or Page Table is loaded in main memory.

- **The pte_clear( ), pmd_clear( ), pud_clear( ), and pgd_clear( ) macros** clear an entry of the corresponding page table.

Linear Address Fields

- PTRS_PER_PTE, PTRS_PER_PMD, PTRS_PER_PUD, and PTRS_PER_PGD
- Compute the number of entries in the Page Table, Page Middle Directory, Page Upper Directory, and Page Global Directory.
- They yield the values 1,024, 1, 1, and 1,024, respectively, when PAE is disabled; and the values 512, 512, 1, and 4, respectively, when PAE is enabled.

Page Table Handling

- No pte_bad( ) macro is defined because it is legal for a Page Table entry to refer to a page that is not present in main memory, not writable, or not accessible at all.
- Instead, several functions are offered to query the current value of any of the flags included in a Page Table entry:
  - pte_read( ) Returns the value of the User/Supervisor flag (indicating whether the page is accessible in User Mode).
  - pte_write( ) Returns 1 if both the Present and Read/Write flags are set (indicating whether the page is present and writable).
  - pte_exec( ) Returns the value of the User/Supervisor flag (indicating whether the page is accessible in User Mode).
- Notice that pages on the Intel processor cannot be protected against code execution.

Page Table Handling

- pte_dirty( ) Returns the value of the Dirty flag (indicating whether or not the page has been modified).
- pte_young( ) Returns the value of the Accessed flag (indicating whether the page has been accessed).
- Another group of functions sets the value of the flags in a Page Table entry:
- pte_wrprotect( ) Clears the Read/Write flag
- pte_rdprotect and pte_exprotect() Clear the User/Supervisor flag
- pte_mkwrite( ) Sets the Read/Write flag
- pte_mkread( ) and pte_mkexec( ) Set the User/Supervisor flag
- pte_mkdirty( ) and pte_mkclean( ) Set the Dirty flag to 1 and to 0, respectively, thus marking the page as modified or unmodified

Page Table Handling

- pte_mkyoung( ) and pte_mkold( ) Set the Accessed flag to 1 and to 0, respectively, thus marking the page as accessed (young) or nonaccessed (old)
- pte_modify(p,v) Sets all access rights in a Page Table entry p to a specified value v
- set_pte Writes a specified value into a Page Table entry
- The macros that combine a page address and a group of protection flags into a 32bit page entry or perform the reverse operation of extracting the page address from a page table entry are:
- mk_ pte( ) Combines a linear address and a group of access rights to create a 32-bit Page Table entry.
- mk_ pte_ phys Creates a Page Table entry by combining the physical address and the access rights of the page

Page Table Handling

- **pte_ page( ) and pmd_ page( )** Return the linear address of a page from its Page Table entry, and of a Page Table from its Page Middle Directory entry.
- **pgd_offset(p,a)** Receives as parameters a memory descriptor p and a linear address a.
- The macro yields the address of the entry in a Page Global Directory that corresponds to the address a; the Page Global Directory is found through a pointer within the memory descriptor p.
- **pgd_offset_k(o)** macro is similar, except that it refers to the memory descriptor used by kernel threads
- **pmd_offset(p,a)** Receives as parameter a Page Global Directory entry p and a linear address a; it yields the address of the entry corresponding to the address a in the Page Middle Directory referenced by p.
- **pte_offset(p,a)** macro is similar, but p is a Page Middle Directory entry and the macro yields the address of the entry corresponding to a in the Page Table referenced by p.

Linear Address Fields

- The macros pmd_bad( ), pud_bad( ) and pgd_bad( ) are used by functions to check Page Middle Directory, Page Upper Directory and Page Global Directory entries passed as input parameters.

- Each macro yields the value 1 if the entry points to a bad page table, that is, if at least one of the following conditions applies:
  - The page is not in main memory (Present flag cleared).
  - The page allows only Read access (Read/Write flag cleared).
  - Either Accessed or Dirty is cleared (Linux always forces these flags to be set for every existing page table).

Page Table Handling

Creation and Deletion of Page Table Entries

- When two-level paging is used, creating or deleting a Page Middle Directory entry is trivial.
- Page Middle Directory contains a single entry that points to the subordinate Page Table.
- Thus, the Page Middle Directory entry is the entry within the Page Global Directory, too.
- When dealing with Page Tables, however, creating an entry may be more complex, because the Page Table that is supposed to contain it might not exist.
- In such cases, it is necessary to allocate a new page frame, fill it with zeros, and add the entry.
- If PAE is enabled, the kernel uses three-level paging. When the kernel creates a new Page Global Directory, it also allocates the four corresponding Page Middle Directories; these are freed only when the parent Page Global Directory is released.
- When two or three-level paging is used, the Page Upper Directory entry is always mapped as a single entry within the Page Global Directory.

Page Table Handling

- Each page table is stored in one page frame; moreover, each process makes use of several page tables.
- The allocations and deallocations of page frames are expensive operations.
- Therefore, when the kernel destroys a page table, it adds the corresponding page frame to a software cache.
- When the kernel must allocate a new page table, it takes a page frame contained in the cache; a new page frame is requested from the memory allocator only when the cache is empty.
- The Page Table cache is a simple list of page frames.

Page Table Handling

- The Page Table cache is a simple list of page frames.
- The pte_quicklist macro points to the head of the list, while the first 4 bytes of each page frame in the list are used as a pointer to the next element.
- The Page Global Directory cache is similar, but the head of the list is yielded by the pgd_quicklist macro. Of course, on Intel architecture there is no Page Middle Directory cache.
- Since there is no limit on the size of the page table caches, the kernel must implement a mechanism for shrinking them.
- Therefore, the kernel introduces high and low watermarks, which are stored in the pgt_cache_water array
- check_pgt_cache( ) function checks whether the size of each cache is greater than the high watermark and, if so, deallocates page frames until the cache size reaches the low watermark.
- The check_ pgt_cache( ) is invoked either when the system is idle or when the kernel releases all page tables of some process.

Page Table Handling

- Now comes the last round of functions and macros:

pgd_alloc( )

- Allocates a new Page Global Directory by invoking the get_ pgd_fast( ) function, which takes a page frame from the Page Global Directory cache; if the cache is empty, the page frame is allocated by invoking the get_ pgd_slow( ) function.

pmd_alloc(p,a)

- Defined so three-level paging systems can allocate a new Page Middle Directory for the linear address a. On Intel 80x86 systems, the function simply returns the input parameter p, that is, the address of the entry in the Page Global Directory.

Page Table Handling

pte_alloc(p,a)

- **Receives as parameters the address of a Page Middle Directory entry p and a linear address a, and it returns the address of the Page Table entry corresponding to a.**
- **If the Page Middle Directory entry is null, the function must allocate a new Page Table.**
- **To accomplish this, it looks for a free page frame in the Page Table cache by invoking the get_ pte_fast( ) function.**
- **If the cache is empty, the page frame is allocated by invoking get_ pte_slow( ).**
- **If a new Page Table is allocated, the entry corresponding to a is initialized and the User/Supervisor flag is set.**
- **pte_alloc_kernel( ) is similar, except that it invokes the get_ pte_kernel_slow( ) function instead of get_ pte_slow( ) for allocating a new page frame; the get_pte_kernel_slow( ) function clears the User/Supervisor flag of the new Page Table.**

Page Table Handling

pte_free( ) , pte_free_kernel( ) , and pgd_free( )

- Release a page table and insert the freed page frame in the proper cache.
- The **pmd_free( ) and pmd_free_kernel( )** functions do nothing, since Page Middle Directories do not really exist on Intel 80x86 systems.
- **free_one_pmd( )** **Invokes pte_free( )** to release a Page Table.
- **free_one_ pgd( )** Releases all Page Tables of a Page Middle Directory; in the Intel architecture, it just invokes **free_one_ pmd( )** once.
- Then it releases the Page Middle Directory by invoking **pmd_free( ).**

Page Table Handling

SET_PAGE_DIR

- Sets the Page Global Directory of a process. This is accomplished by placing the physical address of the Page Global Directory in a field of the TSS segment of the process; this address is loaded in the cr3 register every time the process starts or resumes its execution on the CPU. Of course, if the affected process is currently in execution, the macro also directly changes the cr3 register value so that the change takes effect right away.

new_ page_tables( )

- Allocates the Page Global Directory and all the Page Tables needed to set up a process address space. It also invokes SET_PAGE_DIR in order to assign the new Page Global Directory to the process.

clear_ page_tables( )

- Clears the contents of the page tables of a process by iteratively invoking free_one_ pgd( ).

free_page_tables( )

- Is very similar to clear_ page_tables( ) , but it also releases the Page Global Directory of the process

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 2 of Text Book

Text Book

- **Understanding the Linux Kernel**
  - **Daniel P/ Bovet and Marco Cesati**
    Edition, O' Reilly Media 2005, ISBN 978-0596005658

SOC 3010
OPERATING SYSTEM

END OF

WEEK 4 LECTURE 2

PAGING UNIT

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT
FALL SEMESTER 2024

SOC 3010
OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010
OPERATING SYSTEM

COMPILING & BUILDING LINUX KERNEL FROM SOURCE

SOC 3010
OPERATING SYSTEM

PLEASE NOTE THAT DEBIAN SCRIPT IS USED HERE FOR RECOMPILING & BUILDING LINUX KERNEL FROM SOURCE(NOW NEW SCRIPT IS AVAILABLE AT THE DEBIAN SITE).

YOU CAN ALSO USE LATEST SCRIPT AVAILABLE AT GITHUB TO RECOMPILE & BUILD LINUX KERNEL FROM SOURCE.

Compile & Build Linux Kernal from Source

STEPS:

$cd linux-source-4.4.0

$sudo apt-get install libncurses5 libncurses5-dev

$make menuconfig

$sudo apt-get update && sudo apt-get install libssl-dev

$make Clean

$make deb-pkg

$cd ..

$ls *.deb

Compile & Build Linux Kernal from Source

STEPS:

***Testing the new Kernel**

$sudo dpkg -i linux*4.4.16_4.4.16-1*.deb

$sudo reboot

SOC 3010
OPERATING SYSTEM

END OF

COMPILING & BUILDING LINUX KERNEL FROM SOURCE

PLEASE NOTE THAT DEBIAN SCRIPT IS USED HERE FOR RECOMPILING & BUILDING LINUX KERNEL FROM SOURCE.

YOU CAN ALSO USE LATEST SCRIPT AVAILABLE AT GITHUB TO RECOMPILE & BUILD LINUX KERNEL FROM SOURCE.

INHA UNIVERSITY TASHKENT
FALL SEMESTER 2024

SOC 3010
OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG